locking, in place of the corresponding `std::mutex` specializations. These ensure exclusive access, just as with `std::mutex`. Those threads that don't need to update the data structure can instead use `boost::shared_lock<boost::shared_mutex>` to obtain *shared* access. This is used just the same as `std::unique_lock`, except that multiple threads may have a shared lock on the same `boost::shared_mutex` at the same time. The only constraint is that if any thread has a shared lock, a thread that tries to acquire an exclusive lock will block until all other threads have relinquished their locks, and likewise if any thread has an exclusive lock, no other thread may acquire a shared or exclusive lock until the first thread has relinquished its lock.

The following listing shows a simple DNS cache like the one just described, using a `std::map` to hold the cached data, protected using a `boost::shared_mutex`.

**Listing 3.13. Protecting a data structure with a `boost::shared_mutex`**

```
#include <map>
#include <string>
#include <mutex>
#include <boost/thread/shared_mutex.hpp>

class dns_entry;

class dns_cache
{
    std::map<std::string,dns_entry> entries;
    mutable boost::shared_mutex entry_mutex;
public:
    dns_entry find_entry(std::string const& domain) const
    {
        boost::shared_lock<boost::shared_mutex> lk(entry_mutex);     <-- (1)
        std::map<std::string,dns_entry>::const_iterator const it=
            entries.find(domain);
        return (it==entries.end())?dns_entry():it->second;
    }
    void update_or_add_entry(std::string const& domain,
                             dns_entry const& dns_details)
    {
        std::lock_guard<boost::shared_mutex> lk(entry_mutex);       <-- (2)
        entries[domain]=dns_details;
    }
};
```